

Astro 142

Discussion 13

2025-11-13
Dyson Lewis

Discussion Activity

Discussion 13 Activity

Get the script from Bruinlearn. Follow the instructions.

Paste screenshots into the slides. Upload your final pdf to the discussion assignment

If you have extra time, work on the project! Ask questions!

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.optimize import least_squares
4 from astropy.modeling.functional_models import AiryDisk2D, Gaussian2D
5 from astropy.io import ascii
6
7 import os
8
9 def logparabola(x, phi0, a, b):
10     E0 = 1.0 # normalization at 1 TeV
11     return phi0 * (x / E0) ** (a - b * np.log10(x / E0))
12
13
14 def fit_crab_spectrum(filename):
15     # Make it always look in the same directory as this script
16     script_dir = os.path.dirname(os.path.abspath(__file__))
17     filepath = os.path.join(script_dir, filename)
18
19     data = ascii.read(filepath, format='ecsv')
20     E = data['e_ref'].data
21     dnnde = data['dnnde'].data
22     err = (data['dnnde_errn'].data + data['dnnde_errp'].data) / 2 # sym
23
24     # Residual function for least squares
25     def residuals(params):
26         phi0, a, b = params
27         model = logparabola(E, phi0, a, b)
28         return (dnnde - model) / err
29
30     # Initial guess
31     p0 = [1e-10, -3, 0.1]
32
33     # Fit using least squares
34     result = least_squares(residuals, p0, jac='2-point')
35
36     phi0, a, b = result.x
37     print("Fit results:")
38     print(f"phi0 = {phi0:.3e}")
39     print(f"a = {a:.3f}")
40     print(f"b = {b:.3f}")

```

```

42 # Estimate parameter uncertainties from covariance matrix
43 _, s, VT = np.linalg.svd(result.jac, full_matrices=False)
44 threshold = np.finfo(float).eps * max(result.jac.shape) * s[0]
45 s = s[s > threshold]
46 cov = VT.T @ np.diag(1 / s**2) @ VT
47 perr = np.sqrt(np.diag(cov))
48 print("\nParameter errors (1σ):")
49 print(f"phi0 = {perr[0]:.3e}")
50 print(f"a = {perr[1]:.3f}")
51 print(f"b = {perr[2]:.3f}")
52
53 # Plot data and fit
54 E_fit = np.logspace(np.log10(min(E)), np.log10(max(E)), 200)
55 model_fit = logparabola(E_fit, *result.x)
56
57 plt.errorbar(E, dnnde, yerr=err, fmt='o', label='Data', color='k')
58 plt.plot(E_fit, model_fit, label='Best-fit logparabola', color='red')
59 plt.xscale('log')
60 plt.yscale('log')
61 plt.xlabel('Energy [TeV]')
62 plt.ylabel('dN/dE [cm⁻² s⁻¹ TeV⁻¹]')
63 plt.legend()
64 plt.title('Crab Spectrum Fit (Log-Parabola)')
65 plt.show()
66
67 return result
68
69
70 def fit_psf(model_type='Airy'):
71     xx, yy = np.meshgrid(np.arange(512), np.arange(512))
72     true_model = AiryDisk2D(amplitude=100, x_0=246.3, y_0=230.8, radius=25)
73     clean_data = true_model(xx, yy)
74     noise = np.random.normal(scale=np.sqrt(clean_data.flatten()), size=512**2).reshape((512, 512))
75     noisy_data = clean_data + noise
76
77     plt.imshow(noisy_data, origin='lower', cmap='inferno')
78     plt.title('Noisy PSF Data')
79     plt.colorbar(label='Counts')
80     plt.show()
81

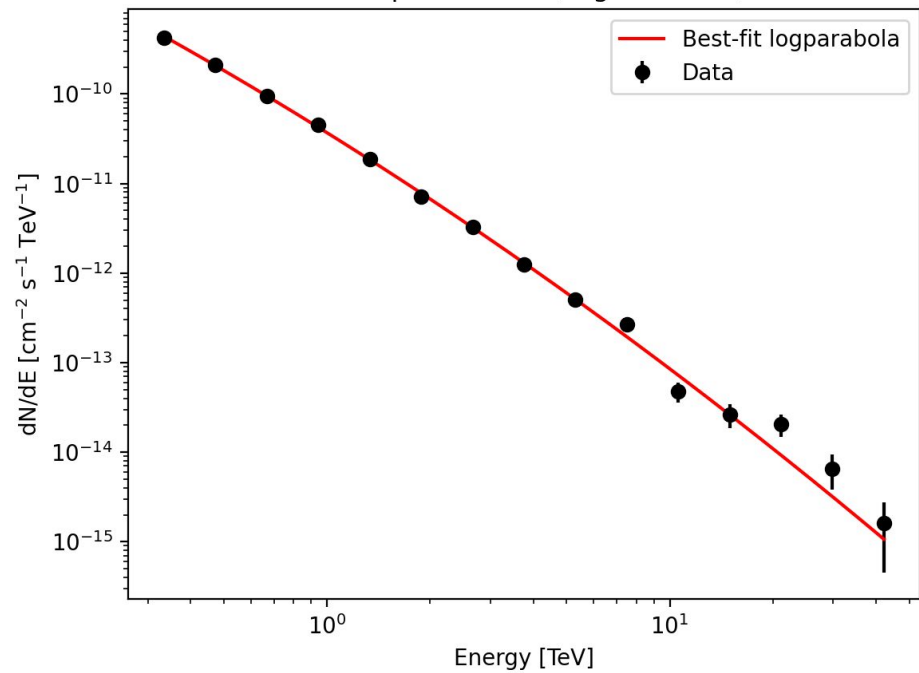
```

```

82 # Residual function to fit either Airy or Gaussian
83 def residual_2d(params):
84     if model_type == 'Airy':
85         amp, x0, y0, radius = params
86         model = AiryDisk2D(amplitude=amp, x_0=x0, y_0=y0, radius=radius)
87     elif model_type == 'Gaussian':
88         amp, x0, y0, sigma_x, sigma_y = params
89         model = Gaussian2D(amplitude=amp, x_mean=x0, y_mean=y0, x_stddev=sigma_x, y_stddev=sigma_y)
90     else:
91         raise ValueError("model_type must be 'Airy' or 'Gaussian'")
92     return (model(xx, yy) - noisy_data).ravel()
93
94 # Initial guesses
95 if model_type == 'Airy':
96     p0 = [90, 250, 230, 20]
97 else:
98     p0 = [90, 250, 230, 10, 10]
99
100 result = least_squares(residual_2d, p0)
101 print(f"\n2D {model_type} Fit Results:")
102 print(result.x)
103
104 # Plot fitted model
105 if model_type == 'Airy':
106     fitted = AiryDisk2D(amplitude=result.x[0], x_0=result.x[1], y_0=result.x[2], radius=result.x[3])
107 else:
108     fitted = Gaussian2D(amplitude=result.x[0], x_mean=result.x[1], y_mean=result.x[2],
109                        x_stddev=result.x[3], y_stddev=result.x[4])
110
111 plt.figure(figsize=(10, 4))
112 plt.subplot(1, 3, 1)
113 plt.imshow(noisy_data, origin='lower', cmap='inferno')
114 plt.title('Data')
115 plt.subplot(1, 3, 2)
116 plt.imshow(fitted(xx, yy), origin='lower', cmap='inferno')
117 plt.title(f'Fitted {model_type}')
118 plt.subplot(1, 3, 3)
119 plt.imshow(noisy_data - fitted(xx, yy), origin='lower', cmap='bwr')
120 plt.title('Residuals')
121 plt.tight_layout()
122 plt.show()

```

Crab Spectrum Fit (Log-Parabola)



Noisy PSF Data

